Guided Exercise 2b

Software Development UC3M 18179

Team Number: 04

Ryan Michael Nuera and Duru Onculoglu

24/04/2026

1. Introduction

This document presents the design and implementation of the test cases for the *Register Document* functionality (`register_document`) of the Enterprise Project Management component developed in Guided Exercise 2b.

The purpose of this work is to verify that the method correctly validates, processes, and stores project documentation based on the requirements provided. The method `register_document(input_file: str)` receives the path to a JSON file, checks its structure and content, and returns a SHA-256 signature, and stores the resulting information in an `all_documents.json` file.

To ensure proper validation of the system, two testing techniques have been applied:

- **Syntactic Analysis**: Test cases have been derived from the formal grammar of the input JSON structure. This allows systematic generation of both valid and invalid inputs, ensuring that the method correctly handles different combinations of structure and data.
- **Structural Testing**: The internal logic of the method has been analyzed using a control flow graph. Based on this analysis, a set of basic execution paths has been identified to ensure that all relevant branches of the code are tested.

The test cases have been defined in an Excel file and executed automatically using Python's `unittest` framework. The testing implementation reads the test definitions, generates the corresponding input files, and verifies the expected outputs or exceptions.

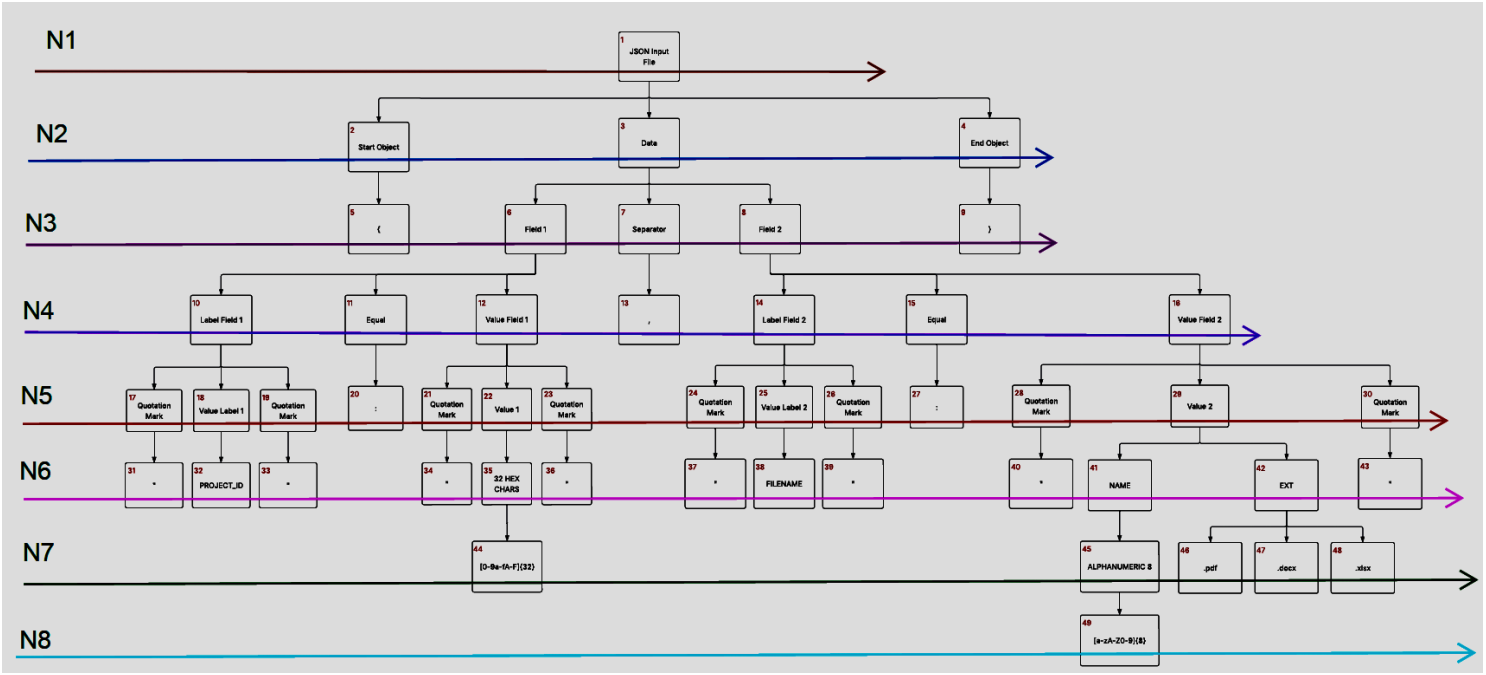
This document includes the grammar used for syntactic analysis, a derivation tree for representative cases, the control flow graph of the method, and the definition of basic paths and loop-related test cases.

1.1 Grammar Definition

JSON Input File ::= Start_Object Data End_Object
Start _Object ::= {
End_Object ::= }
Data ::= Field1 Separator Field 2
Field_1 ::= Label_Field_1 Equal Value_Field_1
Field_2 ::= Label_Field_2 Equal Value_Field_2
Separator ::= ,
Equal ::= :
Quotation Mark ::= "
Label_Field_1 ::= Quotation Mark Value_Label_1 Quotation Mark
Value_Label_1 ::= PROJECT_ID
Value_Field_1 ::= Quotation Mark Value_1 Quotation Mark

Value_1 ::= 32 hex chars
32 chars ::= [0-9a-fA-F]{32}
Label_Field_2 ::= Quotation Mark Value_Label_2 Quotation Mark
Value_Label_2 ::= FILENAME
Value_Field_2 ::= Quotation Mark Value_2 Quotation Mark
Value_2 ::= <NAME><EXT>
NAME ::= alphanumeric 8
alphanumeric 8 ::= [a-zA-Z0-9]{8}
EXT ::= ".pdf" | ".docx" | ".xlsx"

1.2 Derivation Tree



1.3 Design of Test Cases

The test cases have been designed using the syntactic analysis technique, based on the grammar defined previously. Each test case corresponds to modifications applied to nodes of the derivation tree, ensuring coverage of both valid and invalid inputs.

Test Case Categories

- **Valid cases:** Correct JSON structure with valid PROJECT_ID and FILENAME. The expected result is a valid SHA-256 string.
- **Structural errors:** Invalid JSON format due to missing, duplicated, or modified symbols such as {, }, :, and ,.
- **Field errors:** Missing, duplicated, or incorrect field labels (e.g., "PROJECT_NAME" instead of "PROJECT_ID").
- **Value errors:**

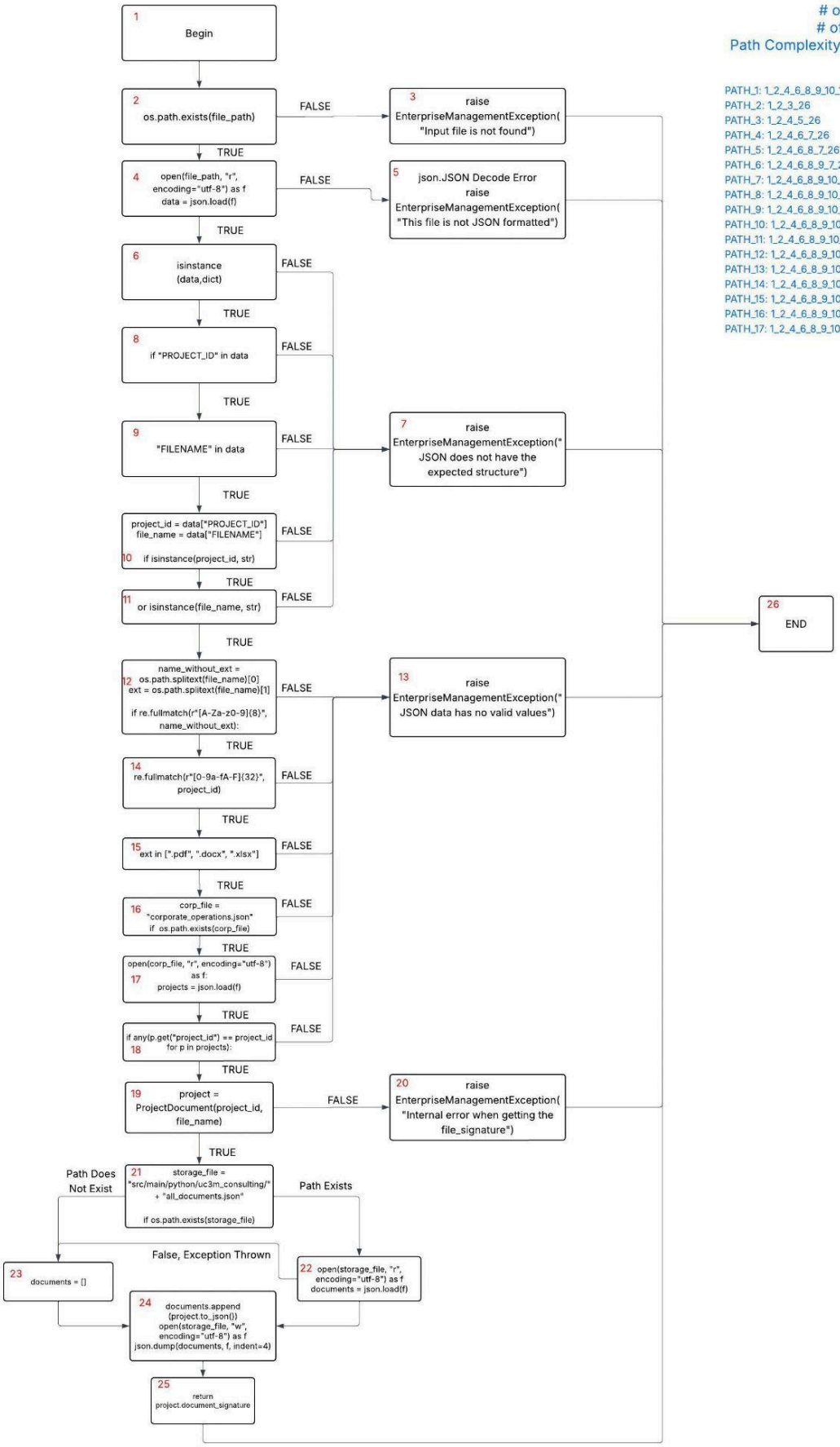
    - PROJECT_ID: must be 32 hexadecimal characters
    - FILENAME: must contain 8 alphanumeric characters + valid extension (.pdf, .docx, .xlsx)
- **Syntax errors:** Incorrect use of quotation marks or separators.

The test cases ensure coverage of all grammar rules by applying deletion, duplication, and modification to both terminal and non-terminal nodes.

Category	Nodes Affected	Operation	Description	Input Condition	Expected Result
Valid Inputs	Multiple	VALID	Correct JSON with supported file extensions (.pdf, .docx, .xlsx)	Well-formed JSON with valid PROJECT_ID and FILENAME	Successful hash output
File-Level Errors	Root	DEL / DUP	Missing or duplicated JSON file	File absent or duplicated content	File not found / Invalid JSON format
Structural JSON Errors	Object delimiters { }	DEL/DUP/ MOD	Corrupted JSON structure (missing, duplicated, or altered braces)	Malformed object boundaries	Invalid JSON format
Empty/ Missing Data	Data node	DEL	JSON exists but contains no data	{}	Invalid structure
Field Presence Errors	Fields (PROJECT_ID, FILENAME)	DEL / DUP	Missing or duplicated fields	Incomplete or repeated fields	Invalid JSON format
Syntax Errors (Separators)	,	DEL/DUP/ MOD	Incorrect or missing field separators	Missing, duplicated, or replaced commas	Invalid JSON format

Syntax Errors (Key-Value Mapping)	:	DEL/DUP/ MOD	Invalid key-value separator	Missing, duplicated, or replaced colon	Invalid JSON format
Label Errors (Keys)	Field names	DEL/DUP/ MOD	Missing, duplicated, or incorrect field names	Invalid or altered keys (e.g., PROJECT_NAME)	Invalid structure
Value Errors (General)	Values	DEL / DUP	Missing or duplicated values	Empty or repeated values	Invalid JSON format / Invalid values
Quotation Errors	"	DEL/DUP/ MOD	Missing, duplicated, or replaced quotation marks	Broken string delimiters	Invalid JSON format
PROJECT_ID Format Errors	PROJECT_ID value	DEL/DUP/ MOD	Invalid hexadecimal format	Empty, duplicated, or non-hex characters	Invalid data values
FILENAME Value Errors	File name	DEL/DUP/ MOD	Invalid filename content	Missing name, duplication, or special characters	Invalid data values
File Extension Errors	Extension	DEL/DUP/ MOD	Invalid or unsupported file extensions	Missing, duplicated, or changed extension	Invalid data values

1.4 The flow control graph (Structural Testing)



# of Edges: 41
# of Nodes: 26
Path Complexity: 41 - 26 + 2 = 17 PATHS

PATH_1: 1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_21_22_24_25_26
PATH_2: 1_2_3_26
PATH_3: 1_2_4_5_26
PATH_4: 1_2_4_6_7_26
PATH_5: 1_2_4_6_8_7_26
PATH_6: 1_2_4_6_8_9_7_26
PATH_7: 1_2_4_6_8_9_10_7_26
PATH_8: 1_2_4_6_8_9_10_11_7_26
PATH_9: 1_2_4_6_8_9_10_11_12_13_26
PATH_10: 1_2_4_6_8_9_10_11_12_14_13_26
PATH_11: 1_2_4_6_8_9_10_11_12_14_15_13_26
PATH_12: 1_2_4_6_8_9_10_11_12_14_15_16_13_26
PATH_13: 1_2_4_6_8_9_10_11_12_14_15_16_17_13_26
PATH_14: 1_2_4_6_8_9_10_11_12_14_15_16_17_18_13_26
PATH_15: 1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_20_26
PATH_16: 1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_21_23_24_25_26
PATH_17: 1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_21_22_23_24_25_26

1.4.1 Definition of the basic paths

PATH	ID TEST	DESCRIPTION
1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_21_22_24_25_26	structuraltest_1	Successful execution path where no exceptions are raised
1_2_3_26	structuraltest_2	Path where the input file is not found
1_2_4_5_26	structuraltest_3	Path where the file is not in valid JSON format
1_2_4_6_7_26	structuraltest_4	Path where the parsed data is not a dictionary instance
1_2_4_6_8_7_26	structuraltest_5	Path where "PROJECT_ID" is missing from the data
1_2_4_6_8_9_7_26	structuraltest_6	Path where "FILENAME" is missing from the data
1_2_4_6_8_9_10_7_26	structuraltest_7	Path where project_id is not a string
1_2_4_6_8_9_10_11_7_26	structuraltest_8	Path where file_name is not a string
1_2_4_6_8_9_10_11_12_13_26	structuraltest_9	Path where file_name is not alphanumeric (8-char req)
1_2_4_6_8_9_10_11_12_14_13_26	structuraltest_10	Path where project_id is not a 32-char hexadecimal string

1_2_4_6_8_9_10_11_12_14_15_13_26	structuraltest_11	Path where file_name does not have a valid extension (.pdf, .docx, or .xlsx)
1_2_4_6_8_9_10_11_12_14_15_16_13_26	structuraltest_12	Path where the corporation file exists to check if project_id is correct.
1_2_4_6_8_9_10_11_12_14_15_16_17_13_26	structuraltest_13	Path where the corporation file is loaded and has a decode or malformed JSON from the corporate file.
1_2_4_6_8_9_10_11_12_14_15_16_17_18_13_26	structuraltest_14	Path where project_id is verified against the corporate operations file to check if it is valid and exists.
1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_20_26	structuraltest_15	Path where an exception is raised during ProjectDocument creation due to failure in retrieving the file signature
1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_21_23_24_25_26	structuraltest_16	Path where the storage file does not exist, so the system falls back before adding the new document.
1_2_4_6_8_9_10_11_12_14_15_16_17_18_19_21_22_23_24_25_26	structuraltest_17	Path where the storage file exists, but an error occurs while reading or parsing it, so the system falls back before adding the new document.

1.4.2 The additional cases required to test the loops

There are no additional test cases required to specifically test loops, as all loops in the implementation are already covered by the existing structural test paths. The designed test cases ensure that each loop is executed at least once and, where applicable, also include scenarios where the loop condition is not satisfied which provides adequate loop coverage.